ZAINA RAHEEN A P (241801328)

```
# Task 1: Import required Python libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
# For encoding categorical data
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

```
# Task 2: Load the Telco Customer Churn Dataset
# Load CSV file into a Pandas DataFrame
df = pd.read_csv("/content/sample_data/WA_Fn-UseC_-Telco-Customer-Churn.csv")

# Display the first 10 records
print("First 10 Records:")
display(df.head(10))

# Display the last 10 records
print("\nLast 10 Records:")
display(df.tail(10))

# Display shape of the dataset
print("\nShape of Dataset:")
print(df.shape)

# Display column names
print("\nColumn Names:")
print(df.columns.tolist())
```

First 10 Records:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	De
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	...	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	...	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	...	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	...	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	...	
5	9305-CDSKC	Female	0	No	No	8	Yes	Yes	Fiber optic	No	...	
6	1452-KIOVK	Male	0	No	Yes	22	Yes	Yes	Fiber optic	No	...	
7	6713-OKOMC	Female	0	No	No	10	No	No phone service	DSL	Yes	...	
8	7892-POOKP	Female	0	Yes	No	28	Yes	Yes	Fiber optic	No	...	
9	6388-TABGU	Male	0	No	Yes	62	Yes	No	DSL	Yes	...	

10 rows × 21 columns

Last 10 Records:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...
7033	9767-FFLEM	Male	0	No	No	38	Yes	No	Fiber optic	No	...
7034	0639-TSIQW	Female	0	No	No	67	Yes	Yes	Fiber optic	Yes	...
7035	8456-QDAVC	Male	0	No	No	19	Yes	No	Fiber optic	No	...
7036	7750-EYXWZ	Female	0	No	No	12	No	No phone service	DSL	No	...
7037	2569-WGERO	Female	0	No	No	72	Yes	No	No	No internet service	...
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	Yes	DSL	Yes	...
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes	Yes	Fiber optic	No	...
7040	4801-JZAZL	Female	0	Yes	Yes	11	No	No phone service	DSL	Yes	...
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	Yes	Fiber optic	No	...
7042	3186-AJIEK	Male	0	No	No	66	Yes	No	Fiber optic	Yes	...

10 rows × 21 columns

```
Shape of Dataset:
(7043, 21)

Column Names:
['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService', 'MultipleLines', 'InternetService', 'Onlin
```

```
# Task 3: Explore the Dataset
# Display information about columns, data types and non-null values
print("Dataset Information:")
df.info()
# include='all' displays statistics for both numerical and categorical columns
print("Summary Statistics:")
display(df.describe(include='all'))
# Find the number of missing values in each column
print("Missing Values in Each Column:")
print(df.isnull().sum())
# Find the number of duplicate records
duplicate_count = df.duplicated().sum()
print("Number of Duplicate Records:", duplicate_count)
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
Summary Statistics:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurit
count	7043	7043	7043.000000	7043	7043	7043.000000	7043	7043	7043	704
unique	7043	2	NaN	2	2	NaN	2	3	3	
top	3186-AJIEK	Male	NaN	No	No	NaN	Yes	No	Fiber optic	N
freq	1	3555	NaN	3641	4933	NaN	6361	3390	3096	349
mean	NaN	NaN	0.162147	NaN	NaN	32.371149	NaN	NaN	NaN	NaN
std	NaN	NaN	0.368612	NaN	NaN	24.559481	NaN	NaN	NaN	NaN
min	NaN	NaN	0.000000	NaN	NaN	0.000000	NaN	NaN	NaN	NaN
25%	NaN	NaN	0.000000	NaN	NaN	9.000000	NaN	NaN	NaN	NaN
50%	NaN	NaN	0.000000	NaN	NaN	29.000000	NaN	NaN	NaN	NaN
75%	NaN	NaN	0.000000	NaN	NaN	55.000000	NaN	NaN	NaN	NaN
max	NaN	NaN	1.000000	NaN	NaN	72.000000	NaN	NaN	NaN	NaN

11 rows × 21 columns

```
Missing Values in Each Column:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
Number of Duplicate Records: 0
```

```
# Task 4(a): Remove duplicate customer records
# Remove duplicate rows
df = df.drop_duplicates()
# Reset the index after removing duplicates
df.reset_index(drop=True, inplace=True)
print("Duplicate records after removal:", df.duplicated().sum())
print("New shape of dataset:", df.shape)

# Task 4(b): Prepare numerical column
# Convert TotalCharges from object/string to numeric
# Invalid values such as blank spaces will become NaN
```

```
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
print("TotalCharges data type:", df['TotalCharges'].dtype)

# Task 4(b): Identify numerical and categorical columns
# Numerical columns
numerical_columns = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
# Categorical columns
categorical_columns = df.select_dtypes(include=['object']).columns.tolist()
print("Numerical Columns:")
print(numerical_columns)
print("\nCategorical Columns:")
print(categorical_columns)
```

```
Duplicate records after removal: 0
New shape of dataset: (7043, 21)
TotalCharges data type: float64
Numerical Columns:
['SeniorCitizen', 'tenure', 'MonthlyCharges', 'TotalCharges']

Categorical Columns:
['customerID', 'gender', 'Partner', 'Dependents', 'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup',
```

```
# Task 4(b)(i): Replace missing numerical values with median
for column in numerical_columns:
    df[column] = df[column].fillna(df[column].median())
print("Missing numerical values replaced using median.")
# Task 4(b)(ii): Replace missing categorical values with mode
for column in categorical_columns:
    df[column] = df[column].fillna(df[column].mode()[0])
print("Missing categorical values replaced using mode.")
# Task 4(c): Verify that no missing values remain
print("Missing values after cleaning:")
print(df.isnull().sum())
# Check total number of missing values
print("\nTotal Missing Values:", df.isnull().sum().sum())
```

```
Missing numerical values replaced using median.
Missing categorical values replaced using mode.
Missing values after cleaning:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64

Total Missing Values: 0
```

```
# Task 5: Encode categorical columns
# Columns specified in the question
encoding_columns = [
    'gender',
    'Partner',
    'Dependents',
    'Contract',
    'PaymentMethod',
    'InternetService',
    'Churn'
]

# Create a LabelEncoder
label_encoder = LabelEncoder()
# Apply label encoding to each selected column
for column in encoding_columns:
    df[column] = label_encoder.fit_transform(df[column])
# Display the encoded columns
print("Encoded Categorical Columns:")
display(df[encoding_columns].head(10))
```

Encoded Categorical Columns:

	gender	Partner	Dependents	Contract	PaymentMethod	InternetService	Churn
0	0	1	0	0	2	0	0
1	1	0	0	1	3	0	0
2	1	0	0	0	3	0	1
3	1	0	0	1	0	0	0
4	0	0	0	0	2	1	1
5	0	0	0	0	2	1	1
6	1	0	1	0	1	1	0
7	0	0	0	0	3	0	0
8	0	1	0	0	2	1	1
9	1	0	1	1	0	0	0

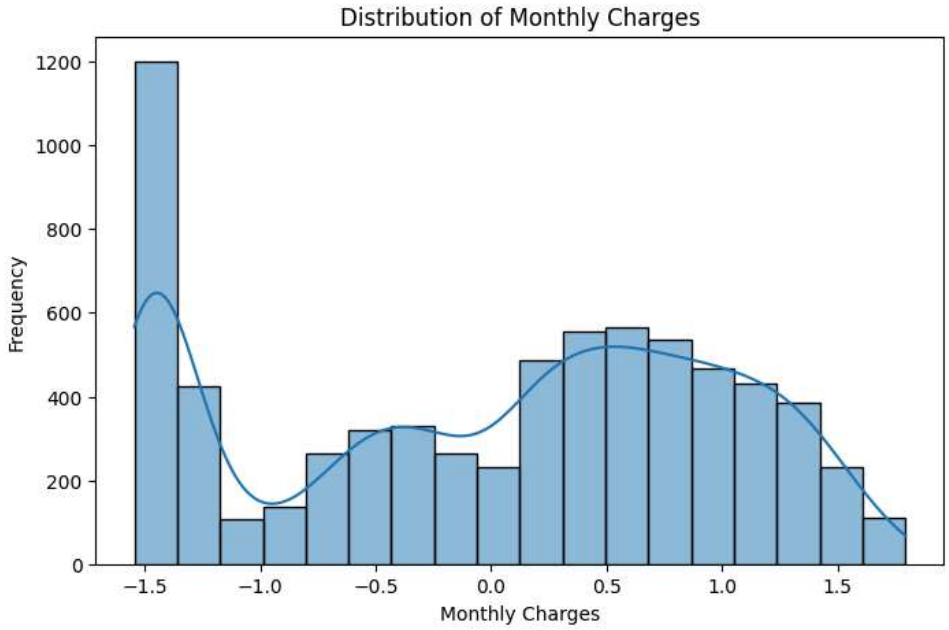
```
# Task 6: Standardise numerical features
# Numerical features specified in the question
scaling_columns = ['tenure', 'MonthlyCharges', 'TotalCharges']
# Create StandardScaler object
scaler = StandardScaler()
# Standardise the selected numerical columns
df[scaling_columns] = scaler.fit_transform(df[scaling_columns])
# Display the standardised values
print("Standardised Numerical Features:")
display(df[scaling_columns].head(10))
```

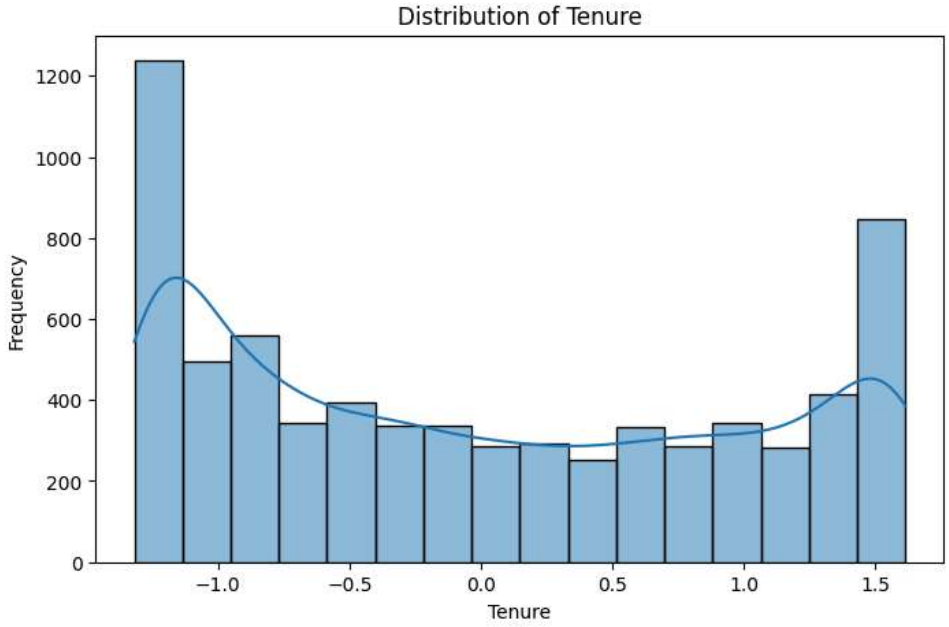
Standardised Numerical Features:

	tenure	MonthlyCharges	TotalCharges
0	-1.277445	-1.160323	-0.994242
1	0.066327	-0.259629	-0.173244
2	-1.236724	-0.362660	-0.959674
3	0.514251	-0.746535	-0.194766
4	-1.236724	0.197365	-0.940470
5	-0.992402	1.159546	-0.645186
6	-0.422317	0.808907	-0.146799
7	-0.910961	-1.163647	-0.874137
8	-0.177995	1.330711	0.337349
9	1.206498	-0.286218	0.532439

```
# Task 7: Plot Distribution of MonthlyCharges
plt.figure(figsize=(8, 5))
sns.histplot(df['MonthlyCharges'], kde=True)
plt.title('Distribution of Monthly Charges')
plt.xlabel('Monthly Charges')
plt.ylabel('Frequency')
plt.show()

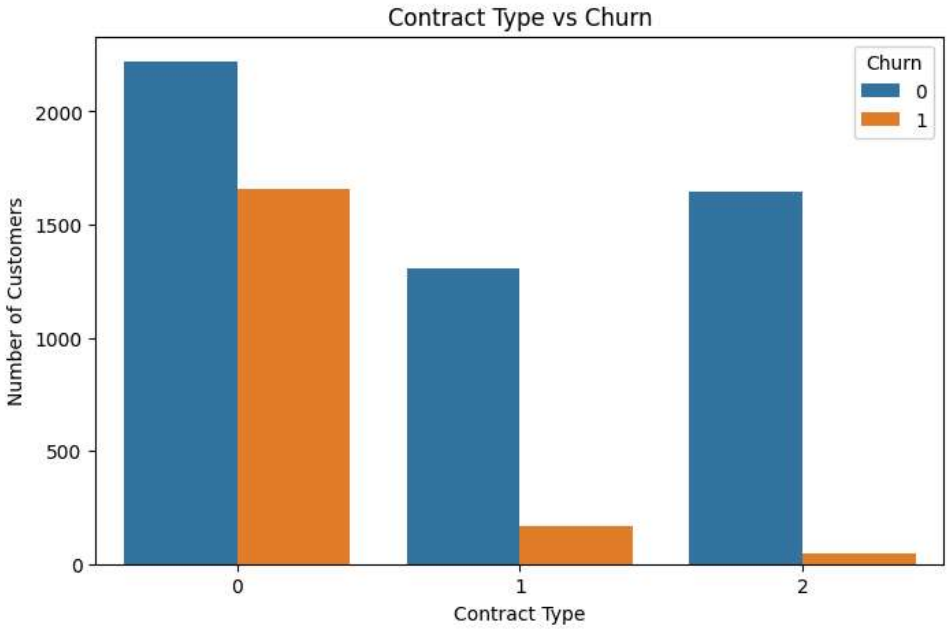
# Task 7: Plot Distribution of Tenure
plt.figure(figsize=(8, 5))
sns.histplot(df['tenure'], kde=True)
plt.title('Distribution of Tenure')
plt.xlabel('Tenure')
plt.ylabel('Frequency')
plt.show()
```

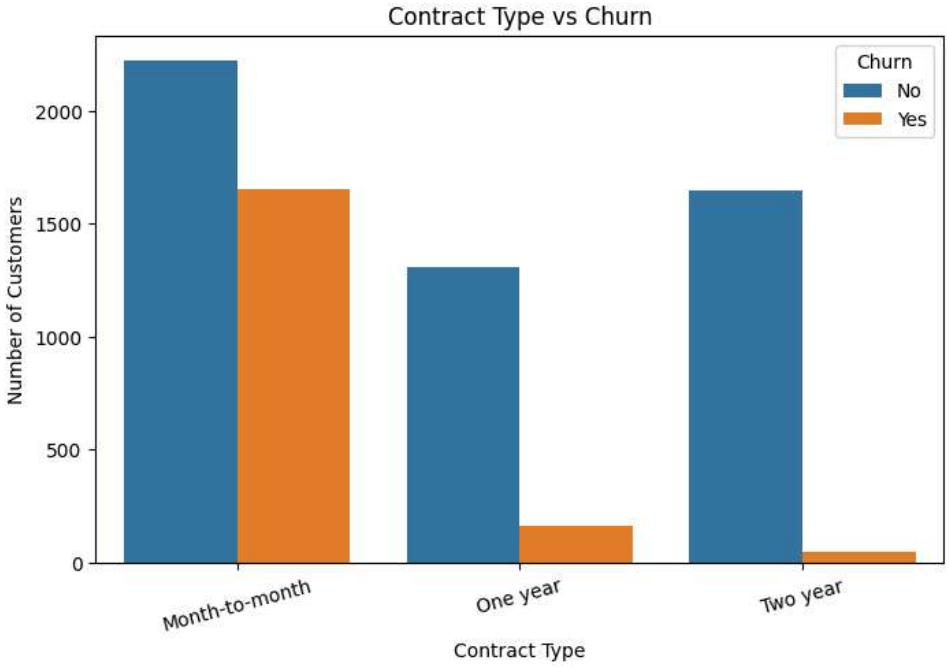





```
# Task 8: Create a Count Plot showing Contract vs Churn
plt.figure(figsize=(8, 5))
sns.countplot(data=df, x='Contract', hue='Churn')
plt.title('Contract Type vs Churn')
plt.xlabel('Contract Type')
plt.ylabel('Number of Customers')
plt.show()

# Task 8: More readable Contract vs Churn plot
# Load a fresh copy of the original dataset for visualization
plot_df = pd.read_csv("/content/sample_data/WA_Fn-UseC_-Telco-Customer-Churn.csv")
plt.figure(figsize=(8, 5))
sns.countplot(data=plot_df, x='Contract', hue='Churn')
plt.title('Contract Type vs Churn')
plt.xlabel('Contract Type')
plt.ylabel('Number of Customers')
plt.xticks(rotation=15)
plt.show()
```






```
# Task 9: Box Plot of MonthlyCharges grouped by Churn
# Load original categorical values for readable labels
plot_df = pd.read_csv("/content/sample_data/WA_Fn-UseC_-Telco-Customer-Churn.csv")
plt.figure(figsize=(7, 5))
sns.boxplot(data=plot_df, x='Churn', y='MonthlyCharges')
plt.title('Monthly Charges Grouped by Churn')
plt.xlabel('Churn')
plt.ylabel('Monthly Charges')
plt.show()
```




```
# Task 10: Display Correlation Heatmap of all numerical features
# Select only numerical columns
numerical_df = df.select_dtypes(include=['int64', 'float64'])
# Calculate correlation matrix
correlation_matrix = numerical_df.corr()
# Plot heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)
plt.title('Correlation Heatmap of Numerical Features')
plt.show()
```

